ИТМО

Как бэкенд обслуживает запросы

Request lifecycle, воркеры, потоки, event loop и async I/O в Python

Санкт-Петербург, 29 сентября 2026 г.

План лекции

1. Жизненный цикл запроса
2. Модель воркеров
3. Блокирующие вызовы
4. Потоки и GIL
5. Event loop и async

Главный вопрос лекции: что происходит с **одним** запросом и что меняется, когда их приходит **сто одновременно**

Часть 1. Жизненный цикл запроса

Вопрос

Мы пишем только функцию-обработчик.

Кто принимает соединение, разбирает HTTP, вызывает наш код и превращает результат в ответ? И чем в этом отличаются Flask, Django и FastAPI?

Сервер ≠ фреймворк

ИТМО

WSGI — синхронный

Фреймворки: Flask, Django / DRF
Серверы: gunicorn, waitress

Приложение — обычная функция:

```
app(environ, start_response)
```

Один вызов = один запрос от начала до конца.

ASGI — асинхронный

Фреймворки: FastAPI, Starlette
Серверы: uvicorn, hypercorn

Приложение — корутина:

```
async app(scope, receive, send)
```

Пока запрос ждёт, сервер обслуживает другие.

`app.run()` во Flask и `runserver` в Django — серверы только для разработки, в продакшене их заменяют gunicorn или waitress

WSGI: приложение — это функция

```
def app(environ, start_response):
    method = environ["REQUEST_METHOD"]
    path = environ["PATH_INFO"]
    start_response("200 OK",
        [("Content-Type", "text/plain")])
    return [f"{method} {path}".encode()]
```

`waitress-serve module:app` — и это уже работающий сервер

`Flask(__name__)` и Django — такие же функции, внутри них роутинг, `request`, валидация.

1. Сервер принимает соединение и разбирает HTTP
2. Кладёт запрос в словарь `environ`: метод, путь, заголовки
3. Вызывает `app(...)`
4. Приложение возвращает код, заголовки и тело
5. Сервер отправляет ответ

Вызов **синхронный**: пока обработчик ждёт базу, поток занят.

ASGI: приложение — это корутина

```
async def app(scope, receive, send):
    if scope["type"] != "http":
        return
    await send({"type": "http.response.start",
                "status": 200,
                "headers": [(b"content-type",
                             b"text/plain")]})
    await send({"type": "http.response.body",
                "body": b"hello"})
```

```
uvicorn module:app
```

- `scope` — данные запроса, как `environ`
- `receive` / `send` — получить запрос и отправить ответ, кусками
- Каждый `await` — точка, где сервер может переключиться на другой запрос
- Умеет WebSocket и долгие соединения — в WSGI их не сделать
- FastAPI построен на Starlette, а это ASGI-приложение

Серверы: кто есть кто

	Протокол	Как держит запросы
`flask run`, `runserver`	WSGI	поток на запрос
gunicorn	WSGI, ASGI через uvicorn-worker	процессы, в каждом 1 или N потоков
waitress	WSGI	один процесс, пул потоков (4 по умолчанию)
uvicorn	ASGI	event loop, процессы через `--workers`

Чем запускать приложение

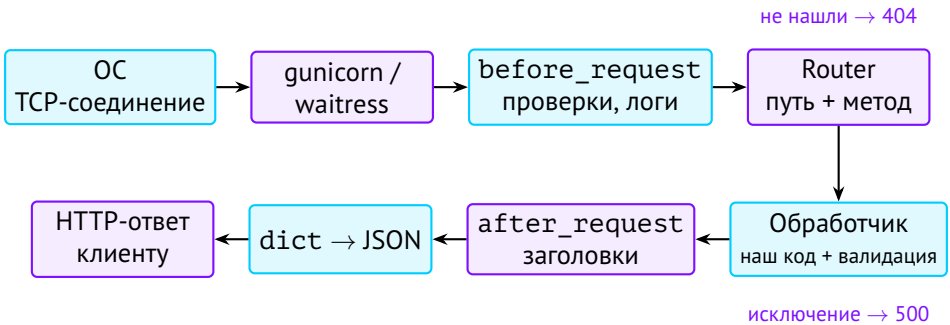
```
flask --app app run --debug                  # Flask, dev
waitress-serve app:app                       # Flask, prod

python manage.py runserver                   # DRF, dev
waitress-serve project.wsgi:application      # DRF, prod

uvicorn main:app --reload                    # FastAPI, dev
uvicorn main:app                             # FastAPI, prod
```

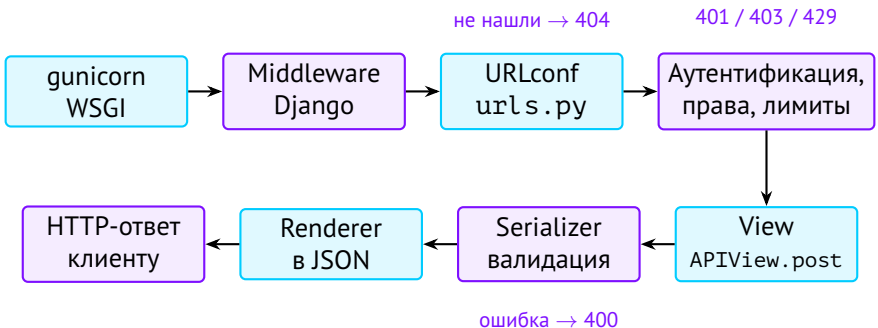
Данные в памяти или в JSON-файле — запускайте **один** процесс, без `-w` и `--workers` (часть 2)

Путь запроса во Flask



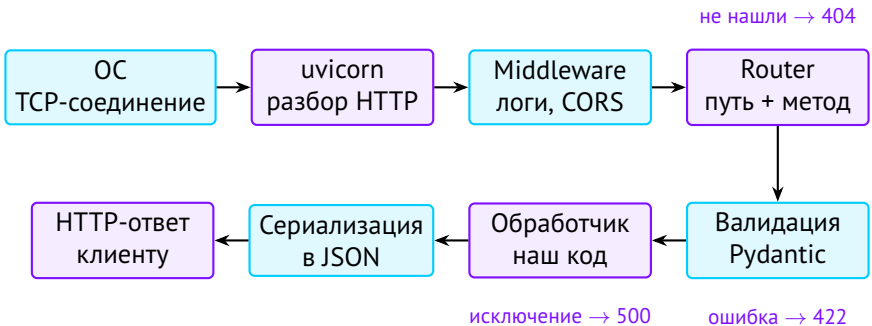
Flask — минимальный фреймворк: валидацию и формат ошибок пишем **сами**.

Путь запроса в Django REST Framework



Django — «всё включено»: ORM, админка, миграции. DRF добавляет к нему REST-слой.

Путь запроса в FastAPI



Всё, кроме блока «Обработчик», за нас делают сервер и фреймворк.

Один эндпоинт: Flask

```
@app.post("/students")
def create():
    data = request.get_json()
    if "isu_id" not in data:
        return {"error": "no isu_id"}, 422
    if repo.exists(data["isu_id"]):
        return {"error": "exists"}, 409
    return service.create(data), 201
```

- Битый JSON — **400**, нет заголовка JSON — **415**: это делает `get_json()`
- Проверку полей пишем сами (или подключаем Pydantic, marshmallow)
- Ответ — кортеж `(тело, код)`

Один эндпоинт: Django REST Framework

```
class StudentIn(serializers.Serializer):
    isu_id = serializers.IntegerField()
    group = serializers.CharField()

class Students(APIView):
    def post(self, request):
        s = StudentIn(data=request.data)
        s.is_valid(raise_exception=True)
        data = s.validated_data
        if repo.exists(data["isu_id"]):
            return Response(
                {"error": "exists"}, 409)
        student = service.create(data)
        return Response(student, 201)
```

- Serializer описывает и проверяет данные
- Ошибка валидации — **400** с описанием каждого поля
- HTTP-методы — методы класса: `get`, `post`, `patch`, `delete`

Один эндпоинт: FastAPI

```
class StudentIn(BaseModel):
    isu_id: int
    group: str

@app.post("/students", status_code=201)
def create(student: StudentIn):
    if repo.exists(student.isu_id):
        raise HTTPException(409, "exists")
    return service.create(student)
```

- **422** — тело не прошло Pydantic, обработчик даже не вызван
- **409** — бизнес-правило, бросаем сами
- **201** — успех, код задан в декораторе
- **500** — любое непойманное исключение

Кто что делает за вас

	Flask	Django + DRF	FastAPI
Протокол	WSGI	WSGI	ASGI
Сервер в проде	gunicorn, waitress	gunicorn, waitress	uvicorn
Валидация	вручную	Serializer	Pydantic по типам
Ошибка валидации	решаете сами	400	422
Параллельность	потоки, процессы	потоки, процессы	event loop + потоки
Размер	микро	«всё включено»	средний

Контекст запроса

ИТМО

Flask

`from flask import request`

`request` выглядит как глобальная переменная, но в каждом запросе она своя: Flask хранит её в **контексте** текущего запроса.

DRF

`def post(self, request)`

Запрос приходит **аргументом** метода. Пользователь — `request.user` после аутентификации.

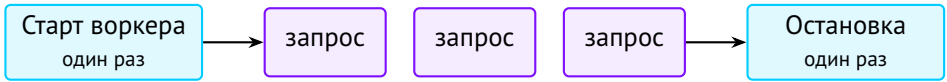
FastAPI

`def get(group: str)`

Фреймворк сам достаёт части запроса и передаёт их **параметрами**. Объект `Request` — если попросить.

Данные одного запроса не должны попасть в другой: не храните их в глобальных переменных модуля

Жизнь приложения и жизнь запроса



загрузить настройки
открыть пул соединений с БД
прочитать JSON-файл

взять соединение из пула
разобрать запрос, ответить
вернуть соединение

закрыть пул
сохранить данные

Дорогое и общее — **один раз на воркер** при старте. Своё для каждого запроса — **на время запроса**, с гарантированным закрытием. При 4 воркерах «старт» выполнится 4 раза.

Старт, запрос, остановка в коде

```
@asynccontextmanager
async def lifespan(app):
    app.state.pool = create_pool()  # start
    yield
    app.state.pool.close()          # stop
def get_conn():
    conn = app.state.pool.acquire() # request
    try: yield conn
    finally: conn.release()

app = FastAPI(lifespan=lifespan)
@app.get("/students")
def students(conn=Depends(get_conn)): ...
```

- Код до `yield` в `lifespan` — перед первым запросом, после — при остановке
- Зависимость с `yield`: `finally` выполнится, даже если обработчик упал
- Flask: старт — код на уровне модуля, на запрос — `g` и `teardown_appcontext`
- Django: старт — `AppConfig.ready()`, соединения с БД открывает и закрывает сам

Часть 2. Модель воркеров

Процесс, поток, корутина

	Процесс	Поток	Корутина
Память	своя, изолирована	общая с процессом	общая с потоком
Кто переключает	ОС	ОС	сам код (`await`)
Стоимость	десятки МБ	мегабайт	килобайт
Все ядра CPU	да	нет (GIL)	нет
В Python	`multiprocessing`, воркеры сервера	`threading`	`asyncio`

Вытесняющая vs кооперативная

Вытесняющая

ОС прерывает задачу в любой момент. Где: процессы и потоки.

+ зависшая задача не вешает остальные
— прервать могут посреди «прочитать — изменить — записать»: гонки, нужен `Lock`

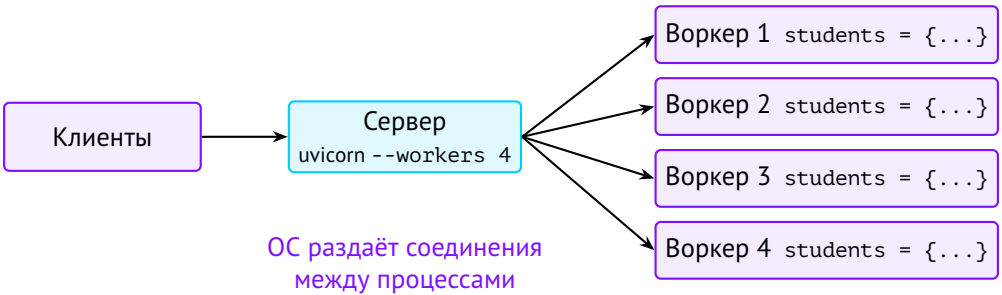
Кооперативная

Задача **сама** отдаёт управление на `await`. Где: `asyncio`, JavaScript.

+ между `await` никто не вмешается
— не отдала управление (`time.sleep`, долгий расчёт) — стоят все

Минус вытесняющей — часть 4, минус кооперативной — часть 5

Воркеры: несколько копий приложения



Каждый воркер — отдельный процесс со своей памятью. Словарь `students` у каждого свой.

Память и 4 воркера

```
students = {}  # per process

@app.post("/students/{isu}")
def create(isu: int):
    students[isu] = {"isu": isu}

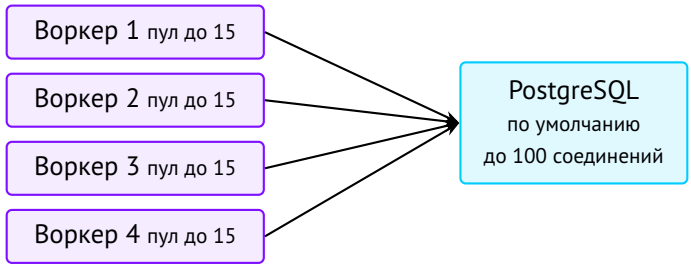
@app.get("/students/{isu}")
def get(isu: int):
    return students[isu]
```

```
POST /students/1 -> worker 2: 201
GET  /students/1 -> worker 3: 404
```

студент сохранён,
но в памяти другого процесса

```
uvicorn app:app --workers 4
```

Пул соединений × воркеры



Пул живёт **в каждом процессе** свой: соединение нельзя передать другому процессу. SQLAlchemy по умолчанию держит до $5 + 10 = 15$ соединений на пул.

8 воркеров × 15 = **120** > 100. Новые соединения база отклонит: `too many clients already`. Считайте: воркеры × размер пула ≤ лимит БД.

Что из этого следует

Хранение в памяти



- Работает только с **одним** воркером
- Перезапуск сервера — данные пропали
- В реальных системах состояние выносят в БД или Redis

Stateless



- Любой воркер должен уметь обработать любой запрос
- Значит, между запросами в воркере ничего важного не храним
- Тогда воркеры можно добавлять без изменений кода

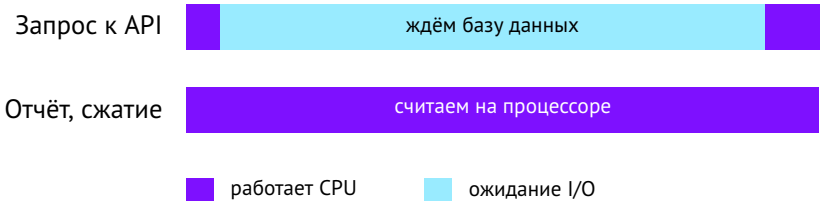
Часть 3. Блокирующие вызовы

Чем занят бэкенд?

Большую часть времени — **ничем**. Он ждёт: ответа базы данных, чтения файла, другого сервиса.

Вопрос в том, что делает сервер, *пока ждёт*.

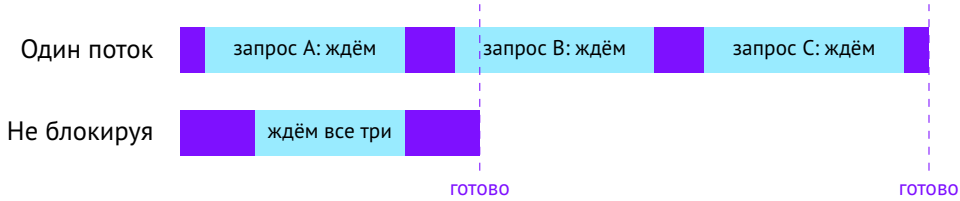
I/O-bound и CPU-bound



I/O-bound — типичный веб. Ускоряем, делая *что-то полезное во время ожидания*.

CPU-bound — редкость в вебе. Ускоряем только *дополнительными ядрами*.

Блокирующий вызов



Блокирующий вызов — поток останавливается и ничего не делает, пока не придёт результат: `time.sleep()`, `requests.get()`, обычный драйвер БД, чтение файла.

Два способа не простаивать: **больше потоков** (часть 4) или **event loop** (часть 5).

Часть 4. Потоки и GIL

Идея

Если один поток ждёт, пусть в это время работает другой.

Так устроены Flask, Django и обычные `def`-обработчики FastAPI.

Потоки в Python

```
from concurrent.futures import (
    ThreadPoolExecutor)

def fetch(url):
    r = requests.get(url)
    return r.status_code

with ThreadPoolExecutor(10) as pool:
    codes = list(pool.map(fetch, urls))
```

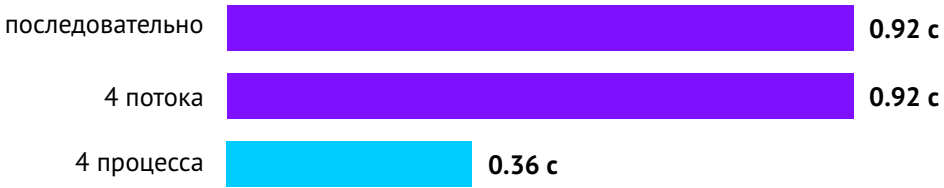
GIL

Global Interpreter Lock: в каждый момент Python-код выполняет только **один** поток процесса.

Пока поток ждёт I/O, GIL отпущен и работают другие.

GIL и CPU-задача

Одна и та же вычислительная задача, пример замера на 4 ядрах



Четыре потока не дали **никакого** ускорения: GIL.

Процессы ускоряют: у каждого свой интерпретатор и свой GIL.

Python без GIL

Что есть сейчас

3.13 (2024) — экспериментальная сборка без GIL (PEP 703)

3.14 (2025) — официально поддерживается (PEP 779), но **не по умолчанию**: отдельная сборка, ставится галочкой в установщике

Проверить: `python -VV` покажет «free-threading build»

Что меняется

+ потоки выполняют Python-код на всех ядрах — CPU-задача в 4 потоках ускоряется

— однопоточный код медленнее на 5–10 %

— C-расширение без поддержки при импорте включает GIL обратно

Гонки никуда не деваются: `Lock` нужен так же.
Веб-серверы пока масштабируются процессами

Гонка при записи в JSON-файл

```
A: read  count = 5
B: read  count = 5
A: write count = 6
B: write count = 6  # lost
```

```
with lock:  # one per repository
    data = json.loads(DB.read_text())
    data["count"] += 1
    DB.write_text(json.dumps(data))
```

- Два потока прочитали одно значение — одно обновление потеряно
- Поток может прочитать файл, записанный наполовину — `JSONDecodeError`, 500
- `Lock` делает «прочитать — изменить — записать» одним шагом

Примитивы синхронизации

Примитив	Что делает	Пример в вебе
`Lock`	в секции только один поток	запись в JSON-файл
`RLock`	тот же поток может взять его повторно	вложенные вызовы с одним локом
`Semaphore(n)`	в секции не больше *n* потоков	не больше 5 запросов к внешнему API
`Event`	«жди, пока не скажут»	ждать, пока загрузится конфиг
`Queue`	безопасная очередь между потоками	фоновый обработчик задач

`Lock` — по сути `Semaphore(1)`. `Queue` часто лучше ручных локов: потоки не делят данные, а передают их друг другу

Semaphore: ограничить одновременность

```
api_limit = threading.Semaphore(5)

def call_api(url):
    with api_limit:     # max 5 at once
        return requests.get(url)
```

- Пришло 100 запросов к нашему бэкенду — во внешний сервис одновременно уйдут только 5
- Остальные ждут на `with`, пока кто-то не выйдет из секции
- Внешний сервис не упадёт и не заблокирует нас за превышение лимита

RLock: лок, который можно взять повторно

ИТМО

```
lock = threading.RLock()

def add_student(s):
    with lock:
        storage[s.isu] = s
        save_all()   # same lock again

def save_all():
    with lock:
        DB.write_text(json.dumps(storage))
```

- С обычным `Lock` поток зависнет навсегда: `save_all` ждёт лок, который держит он сам
- `RLock` помнит, какой поток его держит, и считает захваты. Отпускается, когда счётчик дошёл до нуля
- Другие потоки ждут, как с обычным `Lock`
- Часто лучше без `RLock`: вынести запись в функцию, которая лок не берёт и вызывается только под ним

Event: «жди, пока не скажут»

```
config_ready = threading.Event()

def load_config():    # in a thread
    global config
    config = fetch_config()
    config_ready.set()

@app.get("/students")
def students():
    if not config_ready.wait(timeout=5):
        return {"error": "not ready"}, 503
    ...
```

- `wait()` ждёт, пока кто-то не вызовет `set()`
- Один `set()` будит **всех** ждущих, флаг остаётся поднятым
- `wait(timeout=5)` вернёт `False`, если не дождался
- Ещё применение — остановка: фоновый поток крутится, пока не поднят `stop_event`

Queue: передавать, а не делить

```
tasks = queue.Queue()
def worker():
    while True:
        email = tasks.get()  # waits
        send_email(email)
        tasks.task_done()
Thread(target=worker, daemon=True).start()

@app.post("/register")
def register(user: UserIn):
    save(user)
    tasks.put(user.email)    # no wait
    return {"ok": True}
```

- Обработчик кладёт задачу и **сразу** отвечает, письмо уходит в фоне
- Данные не общие, а передаются — локи писать не нужно, они внутри Queue
- `Queue(maxsize=100)`: при переполнении put ждёт — защита от перегрузки
- Очередь живёт в памяти процесса: при перезапуске задачи пропадут. В проде — Celery или RQ

Deadlock

Поток 1

```
with accounts_lock:
    with logs_lock:
        ...
```

Поток 2

```
with logs_lock:
    with accounts_lock:
        ...
```

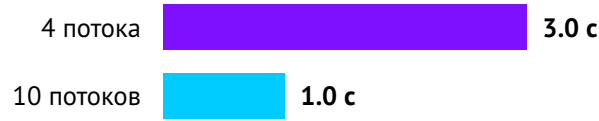
- Каждый поток держит один лок и ждёт второй — ждать они будут **вечно**
- Сервер не падает, а просто перестаёт отвечать, и в логах ничего нет. Поэтому такие ошибки трудно искать
- **Самоблокировка:** поток повторно берёт обычный Lock, который уже держит сам (для этого и есть RLock)

Как не попасть в deadlock

1. Брать локи всегда **в одном и том же порядке**
2. Держать лок как можно меньше: не делать под ним I/O и не вызывать чужой код
3. По возможности обходиться **одним** локом
4. Ставить таймаут: `lock.acquire(timeout=1)` вернёт `False` вместо вечного ожидания

Flask и число потоков

```
@app.get("/slow")
def slow():
    time.sleep(1)   # DB query
    return {"ok": True}
```



10 параллельных запросов по 1 с.
4 потока — три «волны»: 4 + 4 + 2

```
waitress-serve --threads=4
flask_app:app
```

Синхронный сервер обслуживает одновременно **воркеры** × **потоки** запросов. Так работают Flask и Django. `async def` во Flask писать можно, но такой обработчик всё равно занимает целый поток — выигрыша нет.

Часть 5. Event loop и async I/O

Идея

Один поток, который никогда не ждёт: пока один запрос ждёт ответа базы, этот же поток обслуживает другие.

В Python это модуль `asyncio` и синтаксис `async/await`.

Как работает event loop



Переключение происходит **только** на `await`. Код между двумя `await` выполняется без перерыва, никто другой в это время не работает — ни плохо (если там `time.sleep`), ни хорошо (там не бывает гонок).

async/await в Python

```
import asyncio, httpx

async def fetch(client, url):
    r = await client.get(url)
    return r.status_code

async def main():
    async with httpx.AsyncClient() as c:
        return await asyncio.gather(
            fetch(c, "/a"), fetch(c, "/b"))

asyncio.run(main())
```

- `asyncio.run` — запускает цикл (в вебе это делает uvicorn)
- `gather` — как `Promise.all`
- Нужны **асинхронные** библиотеки: `httpx` вместо `requests`
- Отличие от JS: `fetch(c, url)` без `await` ничего не запускает

asyncio: те же примитивы, свои версии

```
lock = asyncio.Lock()

async def withdraw(amount):
    async with lock:
        balance = await get_balance()
        if balance >= amount:
            await set_balance(
                balance - amount)
```

Баланс 100, пять снятий по 100 одновременно: без `lock` прошли все пять, с `lock` — одно

- Гонки бывают через `await`: пока одна корутина ждёт базу, другая успевает прочитать тот же баланс
- `asyncio.Lock`, `Semaphore`, `Event`, `Queue` ждут через `await` и не блокируют цикл
- `threading.Lock` в `async def` ждать нельзя: пока он ждёт, стоит весь event loop
- Deadlock тоже возможен, лечится так же — единым порядком захвата
- `asyncio.Semaphore(10)` ограничивает `gather`: не больше 10 запросов через `httpx` одновременно

Три обработчика в FastAPI

```
@app.get("/sync")
def sync_handler():
    time.sleep(1)            # runs in a thread pool

@app.get("/async-bad")
async def async_bad():
    time.sleep(1)            # blocks the whole event loop

@app.get("/async-good")
async def async_good():
    await asyncio.sleep(1)   # yields to the loop while waiting
```

Один воркер, по 10 параллельных запросов в каждый обработчик. Сколько секунд займёт каждый?

Результат



`async def` сам по себе ничего не ускоряет. Один блокирующий вызов внутри него делает сервер **последовательным**: 10 запросов по секунде — 10 секунд.

Правила для FastAPI

`def`

- Внутри обычные библиотеки: `requests`, файлы, синхронный драйвер БД
- Выполняется в пуле потоков
- Безопасный выбор по умолчанию

`async def`

- Внутри только `await` асинхронных библиотек
- Никаких `time.sleep`, `requests`
- Блокирующее — через `await asyncio.to_thread(f)`

Итоги

Задача	Инструмент	Где встречается
Использовать все ядра	несколько процессов	воркеры gunicorn / uvicorn
Ждать I/O, код синхронный	потоки	Flask, Django, `def` в FastAPI
Ждать много I/O сразу	event loop	`async def` в FastAPI
Общие данные в потоках	`threading.Lock`	запись в JSON-файл

Если данные хранятся в памяти или в JSON-файле

Запускайте с **одним** воркером. Flask, DRF и `def` в FastAPI работают в потоках — при хранении в JSON-файле защищайте запись `Lock`. В `async def` не вызывайте блокирующий код.

Спасибо
за внимание!

ITMO more than a UNIVERSITY